Task 1 — Speech LLM Architecture Comparison

Moshi vs Qwen2.5-Omni

Executive summary

This section compares Moshi (Défossez et al., 2024), a full-duplex speech-text dialogue model, and Qwen2.5-Omni (Xu et al., 2025), a unified multimodal model that accepts audio and other modalities and generates text and natural speech. Both support speech-to-speech interaction, but they organize streaming, semantic content, and acoustic output differently. Moshi is explicitly designed around simultaneous user/system audio streams and conversational overlap. Qwen2.5-Omni separates multimodal understanding/text generation (“Thinker”) from streaming speech-token generation (“Talker”).

1. Moshi

Moshi is a speech-text foundation model and full-duplex spoken dialogue framework. Its design targets limitations of cascaded systems—voice activity detection, ASR, text dialogue, and TTS—where component boundaries add latency and a text bottleneck can discard emotion, prosody, laughter, and other non-verbal signals.

Architecture and representations

Moshi uses a text-language-model backbone augmented with an audio language model that processes and predicts discrete audio units produced by a neural audio codec. User audio and assistant audio are represented as parallel streams. The system also uses “Inner Monologue”: time-aligned text tokens are predicted before corresponding audio tokens, helping preserve linguistic content while generating speech. The codec representation is hierarchical, separating information useful for semantic content from finer acoustic detail.

Alignment and streaming

The parallel streams make it possible to model interruptions, interjections, overlap, and listening while speaking rather than requiring a strict one-speaker-at-a-time turn boundary. The paper reports a theoretical latency of 160 ms and approximately 200 ms in practice for its demonstrated configuration; these are paper-reported system results, not guarantees for every device, language, or deployment. Streaming speech recognition and text-to-speech are also described.

Generation, stopping, and training

Moshi autoregressively predicts discrete tokens across multiple streams. The text “inner monologue” provides a linguistic scaffold for speech generation. As in other autoregressive decoders, generation ends according to learned stop/end tokens and runtime stream/turn control; a deployed application still needs explicit interruption, timeout, and end-of-turn policies. The paper describes training a speech-text model on conversational data and jointly modeling the user and assistant streams; implementation details and data scale should be taken from the paper rather than inferred from the architecture alone.

Strengths and limitations

Strengths: explicit full-duplex design; low-latency streaming; joint modeling of user and assistant speech; better opportunity to preserve prosody and non-verbal acoustic cues than a text-only bottleneck; supports overlapping speech and interruptions.

Limitations: multi-stream codec-token prediction is technically complex and computationally demanding; acoustic quality depends on codec and data; latency can grow with hardware, chunking, and buffering; reliable barge-in and turn control remain system-level problems; multilingual and low-resource language performance must be measured rather than assumed. A speech-native model may also be harder to inspect than a text transcript pipeline.

2. Qwen2.5-Omni

Qwen2.5-Omni is an end-to-end multimodal model designed to process text, audio, images, and video and to generate text and natural speech in a streaming manner.

Architecture and representations

Its “Thinker-Talker” design divides responsibilities. The Thinker is a Transformer decoder with audio and visual encoders that extracts multimodal information and generates high-level representations and text. The Talker receives high-level representations from the Thinker and generates discrete speech tokens. The Talker is a dual-track autoregressive Transformer decoder. This separation gives the model a semantic/text generation pathway and a speech-generation pathway while allowing the components to be trained and inferred end-to-end.

Alignment and streaming

The report describes block-wise processing in the audio and visual encoders to support streaming inputs. For audio-token decoding, a sliding-window diffusion Transformer (DiT) constrains the receptive field to reduce initial packet delay. The model also introduces TMRoPE (Time-aligned Multimodal RoPE) to synchronize video timestamps with audio; this is especially relevant to audio-video inputs rather than being a speech-only feature.

Generation, stopping, and training

The Talker autoregressively generates speech tokens conditioned on Thinker representations and shared historical context. Text and speech are generated concurrently while reducing interference between modalities. Runtime stopping depends on the model's end-of-sequence/streaming conventions and application control; production systems should implement cancellation, silence/timeout handling, and chunk-boundary rules. The report describes end-to-end training and inference for Thinker and Talker.

Strengths and limitations

Strengths: one unified multimodal model; block-wise input supports streaming; concurrent text and speech generation; separation of understanding/text and speech output makes the roles easier to reason about; supports multimodal extensions beyond audio.

Limitations: more components and modalities can increase memory and deployment complexity; output latency depends on encoder chunk size, generation, and audio decoding; speech-token generation remains autoregressive and can become a bottleneck; video-related machinery is unnecessary overhead for speech-only applications; benchmark results reported by the authors do not automatically transfer to Malayalam or noisy spontaneous conversations.

3. Direct comparison

4. Which design is better for low-latency, natural real-time conversation?

There is no universal winner. Moshi's explicit full-duplex parallel-stream design is especially aligned with interruptions, overlap, and turn-taking. Qwen2.5-Omni offers a broad multimodal interface and a Thinker-Talker split that supports streaming speech generation. In both cases, end-to-end latency includes audio capture, chunking, model inference, decoding, transport, and playback—not just the model's reported theoretical latency.

For an Indic conversational assistant, the main risks are: (1) code-switching and regional accent coverage; (2) noisy/overlapping input; (3) end-of-turn detection and barge-in; (4) generation delay; (5) hallucinations or incorrect answers; and (6) evaluation mismatch between scripted read speech and natural dialogue. Common Voice is read speech, so the ASR experiment in this project does not demonstrate spontaneous conversational robustness.

5. Proposed improvement

I propose a streaming, confidence-aware hybrid pipeline for a resource-constrained Indic assistant:

1. Use a chunked ASR encoder with partial hypotheses and a lightweight endpoint detector.

2. Preserve the audio-derived transcript and confidence/uncertainty metadata rather than passing only a final string.

3. Retrieve a small, trusted set of facts for factual questions and pass the retrieved evidence to the response model; if retrieval is weak or the transcript is uncertain, ask for clarification or abstain.

4. Generate short text responses first; optionally use streaming TTS for audio output. For lower latency, start synthesis on stable text chunks rather than waiting for a long response.

5. Add explicit end-of-turn, timeout, cancellation, and barge-in policies at the application layer.

6. Evaluate with fixed test sets across clean/noisy audio, speaking rates, speakers, code-switching, and question types; report WER/CER, answer correctness, hallucination/abstention, first-response latency, and real-time factor.

This hybrid is less ambitious than training a speech-native foundation model, but is practical to reproduce on limited hardware. The project's small LLM pilot supports testing retrieval as an intervention: the Qwen2.5-1.5B model had a 0% exact-answer substring match on 10 no-context pilot questions and 60% with supplied context. The latter is only a mechanical screening metric, not human-verified semantic accuracy; it motivates further evaluation rather than proving a general solution.